BTS Services Informatiques aux Organisations
Fiche hebdomadaire de suivi de stage

PARCOURS SISR ☐	**PARCOURS SLAM** ☑

Semaine n° 5 du **02 février** au **6 février** 2026

Nom de l'étudiant : PORTOLLEAU Anaïs

Nom de l'organisation d'accueil : DELIA TECHNOLOGIES

Nom, adresse e-mail et/ou téléphone du tuteur au sein de l'organisation :

Tuteur : Julien Relandeau
Contacter Pauline Francheteau : 06 14 95 06 49

Nom du tuteur école : Ouafâa Laouija / Nicolas Bourgeois

Rappel du contexte *(activité de l'organisation, nom du service d'accueil et rôle, projet éventuel, …)* :

DELIA TECHNOLOGIES est une ESN (Entreprise de Services du Numérique) fondée en 2019 et basée à Nantes. Elle accompagne ses clients dans la conception, le développement et la maintenance de solutions numériques innovantes, en apportant une expertise technique adaptée à leurs besoins. Une seconde ESN a également été créée en 2021 à Rennes.

À la fin de l'année 2025, une nouvelle version majeure de Node.js ainsi qu'une évolution de l'API Pennylane ont été déployées. Cette évolution technologique a rendu nécessaire la mise à jour de plusieurs projets afin de garantir leur compatibilité, leur sécurité et leur bon fonctionnement dans le cadre de leur exploitation en 2026.

Dans ce contexte, j'ai été amenée à intervenir sur une application existante nommée **Lucas 2 Penny**. Il s'agit d'une application métier permettant notamment la gestion des factures, des données internes de l'entreprise ainsi que d'autres fonctionnalités essentielles. Le projet n'ayant pas été migré depuis une mise à jour datant d'environ juin 2025, une migration vers la nouvelle version était indispensable.

Cette application repose sur l'utilisation de **deux API externes nommées Lucca et Pennylane**.

Mon rôle a consisté à contribuer à cette évolution technique en adaptant le code existant, en identifiant les incompatibilités liées à la montée de la version 2 de l'application, ainsi qu'aux évolutions de l'une des API qui est Pennylane.

Compte rendu du travail effectué *(liste des tâches avec leur description)* :

1. Développement de la fonctionnalité du rattachement des factures

Objectif : vérifier que les données sont bien conservées lors du rattachement d'une facture.

- Constat initial : Lors du rattachement d'une facture, le nom du client n'était pas affich (les autres informations, j'avais réussi à les afficher la semaine dernière.
- Analyse :Après inspection de la console (logs), il est apparu que les données client étaient bien présentes mais **non affichées côté interface**.
- J'ai donc effecutués des modifications et voici ce que j'ai modifié :

Fonction :
const pennyInvoices = async () => {

```
const pennyInvoices = async () => {
  try {
    setIsLoadingInvoice(true);
    setNotFoundErrorMessage('');
    setDisplayedInvoice(null);

    console.log('Recherche facture par numéro:', searchId);

    // Récupérer la facture depuis Pennylane
    const invoice = (await fetchPennylaneInvoiceByIdNumber(searchId)) as IPennylaneInvoiceExtended;
    console.log('Facture récupérée depuis Pennylane:', invoice);

    if (!invoice) {
      setNotFoundErrorMessage("La facture n'existe pas");
      return;
    }

    // Récupérer le client complet si possible
    if (invoice.customer?.id) {
      try {
        const fullCustomer = await fetchPennylaneCustomerById(invoice.customer.id.toString());
        invoice.customer = fullCustomer;
        console.log('Client complet récupéré:', fullCustomer);
      } catch (err) {
        console.warn('Impossible de récupérer les détails du client:', err);
      }
    }
```

```
    // Récupérer les lignes de facture depuis le backend
    let lines: IPennylaneInvoiceLineItem[] = [];
    try {
        const res = await fetch(`/api/invoices/invoice-lines/${invoice.id}`);
        if (res.ok) lines = await res.json();
        else console.warn('Impossible de récupérer les lignes depuis le backend');
    } catch (err) {
        console.error(err);
    }
    invoice.invoice_lines = lines;


    // Afficher la facture dans le modal
    setDisplayedInvoice(invoice);

  } catch (err) {
    console.error(err);
    setNotFoundErrorMessage("Erreur lors de la récupération de la facture");
  } finally {
    setIsLoadingInvoice(false);
  }
};
```

Partie affichage du client : {/* Client */} :

```
<div className="w-1/3 text-xs">
  <strong>Client:</strong> {displayedInvoice.customer?.name || '–'}<br />
  {displayedInvoice.customer?.billing_address?.address || '–'}<br />
  {displayedInvoice.customer?.billing_address
    ? `${displayedInvoice.customer.billing_address.postal_code} ${displayedInvoice.customer.billing_address.city}`
    : '–'}<br />
</div>
```

Interface : export interface IPennylaneCustomer :

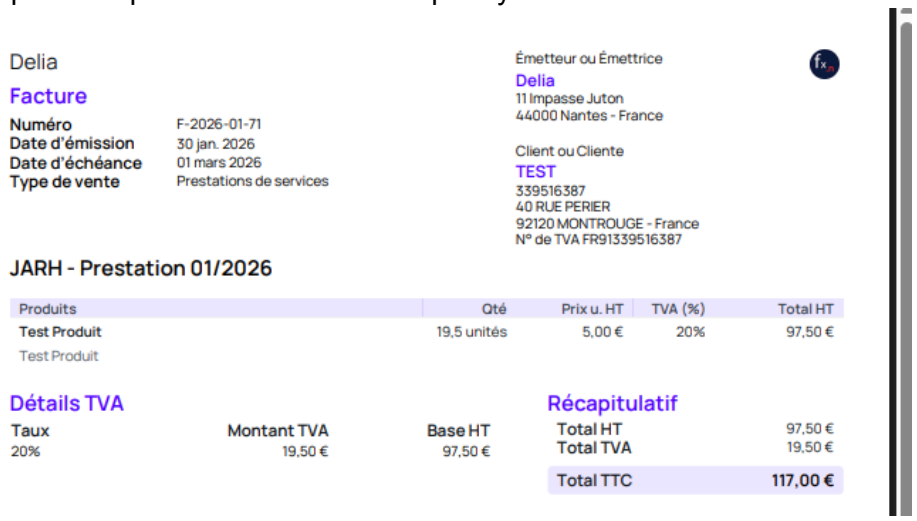
```
/**
 * Client Pennylane
 */
export interface IPennylaneCustomer {
    id: number;
    name: string;
    external_reference?: string;
        billing_address?: {
        address?: string;
        postal_code?: string;
        city?: string;
        country_alpha2?: string;
    };
}
```

<u>Résultat :</u>

Grâce à ces modifications, **le nom du client s'affiche correctement** et correspond bien aux informations présentes dans Pennylane.

Puis on peut voir que grâce à ces modifications cela s'affichequand on fait le test en cliquant sur "Rechercher" :

Et qui correspond bien à celle dans pennylane :



Delia

Facture

Numéro	F-2026-01-71
Date d'émission	30 jan. 2026
Date d'échéance	01 mars 2026
Type de vente	Prestations de services

Émetteur ou Émettrice
Delia
11 Impasse Juton
44000 Nantes - France

Client ou Cliente
TEST
339516387
40 RUE PERIER
92120 MONTROUGE - France
N° de TVA FR91339516387

JARH - Prestation 01/2026

Produits	Qté	Prix u. HT	TVA (%)	Total HT
Test Produit Test Produit	19,5 unités	5,00 €	20%	97,50 €

Détails TVA

Taux	Montant TVA	Base HT
20%	19,50 €	97,50 €

Récapitulatif

Total HT	97,50 €
Total TVA	19,50 €
Total TTC	117,00 €

2. Rattachement de la facture au client

- Tentative de rattachement de la facture après affichage correct du client.J'ai rencontré ce problème:
  - Erreur au clic sur le bouton de rattachement.
  - Erreur également visible dans la modale.

- Correction :
  - Analyse et résolution de l'erreur bloquante.
  - Vérification via des tests.

- Résultat :
  - Les informations s'affichent correctement
  - La facture est **bien rattachée au client**.
  - Le rattachement fonctionne comme attendu.

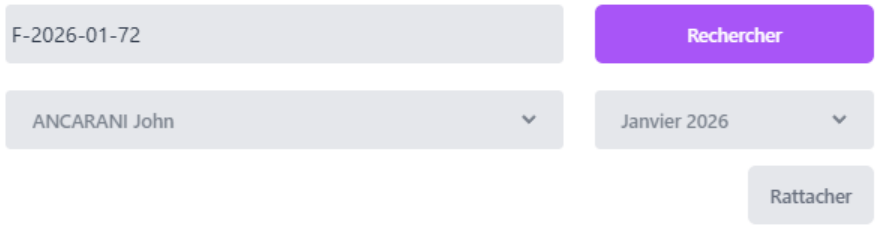
J'ai essayé par la suite de rattacher la facture au client mais j'ai rencontré cette erreur au clic du bouton :

```
Erreur lors de la création de la facture : Error: Impossible de récupérer les settings (status 404)
    at sendInvoiceInfosToBackend (ModalImportInvoice.tsx:145:26)
    at async createInvoice (ModalImportInvoice.tsx:221:7)
```

et sur la modale j'ai cette erreur :

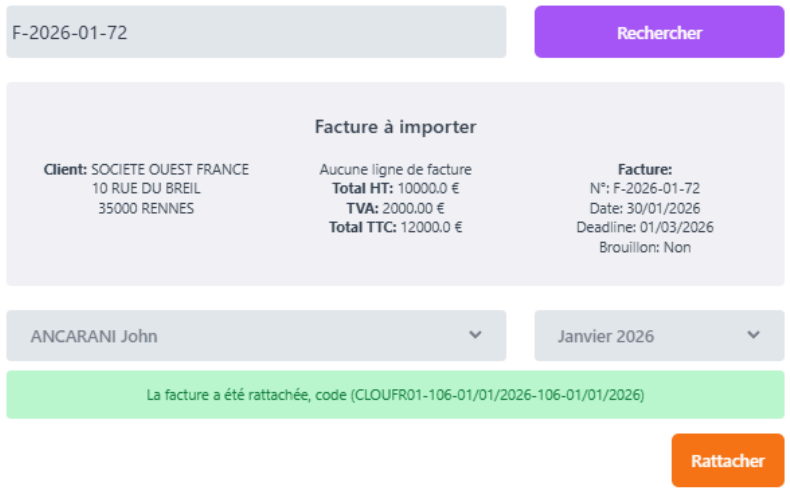
J'ai résolu ensuite cette erreur comme on peut le voir ci-dessous en modifiant la fonction pour importer des factures (car on pouvait pas rattacher une facture où un brouillon existait déjà, qu'on essaie de rattacher une facture finale):

Si on fait le test, cela donne ceci :



Cela affiche bien des infos :

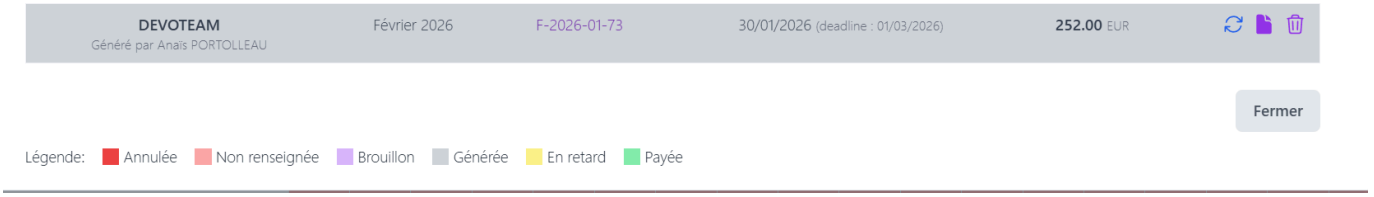
On peut voir qu’elle est bien rattachée :



Mais quand on fait rattacher, on voit ceci dans les logs donc cela veut dire que cela n’a pas bien marché:

```
Recherche facture par numéro: F-2026-01-72                                                    ModalImportInvoice.tsx:77
Facture récupérée depuis Pennylane:                                                           ModalImportInvoice.tsx:81
  ▸ {id: 4080621713, label: 'Facture SOCIETE OUEST FRANCE - F-2026-01-72 (label généré)', invoice_number: 'F-2026-01-72', currency: 'EUR', amount: '12000.0', …}
[fetchPennylaneCustomerById] source_id reçu : 118098366                                       customer.service.ts:19
[fetchPennylaneCustomerById] résultat de la recherche : ▸ {items: Array(1), has_more: false, next_cursor: null}   customer.service.ts:34
Client complet récupéré avec ID correct:                                                      ModalImportInvoice.tsx:110
  ▸ {billing_iban: null, created_at: '2024-07-19T09:36:55.003520Z', customer_type: 'company', id: 118098366, name: 'SOCIETE OUEST FRANCE', …}
✕ ▸ GET http://lucca2penny.localhost/api/invoices/invoice-lines/4080621713 500 (Internal Server Error)   ModalImportInvoice.tsx:123
⚠ ▾ Impossible de récupérer les lignes depuis le backend, status: 500                         ModalImportInvoice.tsx:133
    pennyInvoices @ ModalImportInvoice.tsx:133
    Afficher les frames de la liste des éléments à ignorer
⚠ ▾ Aucune ligne de facture récupérée, vérifiez le backend.                                   ModalImportInvoice.tsx:144
    pennyInvoices @ ModalImportInvoice.tsx:144
    Afficher les frames de la liste des éléments à ignorer
[vite] server connection lost. polling for restart...                                         client.ts:105
✕ ▸ GET http://lucca2penny.localhost/ net::ERR_EMPTY_RESPONSE                                 client.ts:339
```

Après ces analyses de code et des modifications, on peut voir que ça marche après avoir généré :



3. Factures générées

J'ai fait la correction du **bouton "Mise à jour"** dans la page "factures générées". Le problème initial étant que le bouton mise à jour tournait en boucle sans fin.

J'ai modifié cette fonction pour régler le soucis en ajoutant un try/catch pour que cela puisse fonctionner :

```
/**
 * Handler du clic :
 * - passe en mode loading
 * - exécute l'action asynchrone
 * - repasse en mode normal une fois terminé (même en cas d'erreur)
 */
const onButtonClick = async () => {
    setLoading(true);
    try {
        await onClick();
    } catch (error) {
        console.error('Erreur lors de l'exécution de l'action du bouton:', error);
    } finally {
        setLoading(false);
    }
};
```

Et au final pour le résultat, le fonctionnement est rétabli.

4. Bugs identifiés au cours de la semaine

Puis, j'ai fait un état des lieux du site et des bugs qui sont présents qui sont à réparés :

- **Page Saisies journalières** :
    - Passage de l'état *orange* → *vert* à vérifier.

Pour cela j'ai effectué la correction du bouton invoice.route.ts dans le dossier backend/src/server/routes/invoice.route.ts, ce code gère une erreur de contrainte unique : s'il existe déjà une facture avec le même pennylaneId, il la retourne au lieu de créer un doublon. Sinon, il renvoie une erreur au serveur indiquant que la contrainte a été violée.

```
if (error instanceof UniqueConstraintError) {
    console.error(
        '[INVOICE ROUTE] Unique constraint error:',
        error
    );
    // Try to find the existing invoice by pennylaneId
    const existingInvoice = await getInvoiceByPennyaneId(
        req.body.invoice.id
    );
    if (existingInvoice) {
        console.log(
            '[INVOICE ROUTE] Returning existing invoice:',
            existingInvoice.id
        );
        return res.status(status.OK).json(existingInvoice);
    }
    console.error(
        '[INVOICE ROUTE] No existing invoice found for pennylaneId:',
        req.body.invoice.id
    );
    return res
        .status(status.INTERNAL_SERVER_ERROR)
        .json({ error: 'Invoice constraint violation' });
}

return res.status(status.INTERNAL_SERVER_ERROR).json(error);
}
```

Correction dans le fichier Modal, où ce code initialise l'état currentProject avec le premier projet de la liste projects et restaure son status depuis localStorage si une valeur avait été sauvegardée précédemment. Ainsi, l'utilisateur retrouve son dernier état même après un rafraîchissement de page.

```
const [currentProject, setCurrentProjet] = useState<ViewDayItemProject>(() => {
    const project = projects[0];
    // Restore status from localStorage if available
    const savedStatus = localStorage.getItem(`project_${project.code}_status`);
    if (savedStatus) {
        project.status = savedStatus;
    }
    return project;
});
```

Et modif de cette fonction avec l'ajout d'une condition if, Cette fonction gère le changement de projet : si le projet sélectionné (selectedProjectCode) est différent de currentProject, elle cache les détails de la facture, retrouve le projet correspondant dans la liste projects, restaure son status depuis localStorage si

disponible, puis met à jour l'état currentProject et le nombre de jours nbDays

```
function handleProjectChanged() {
    // changement de projet ?
    if (currentProject.code != selectedProjectCode) {
        // on cache les détails de facture
        setShowDetailInvoice(false);
        // récupération du bon projet
        const project: ViewDayItemProject | undefined = projects.find(
            (p) => p.code == selectedProjectCode
        );
        // Projet trouvé ?
        if (project) {
            // Restore status from localStorage if available
            const savedStatus = localStorage.getItem(`project_${project.code}_status`);
            if (savedStatus) {
                project.status = savedStatus;
            }
            setCurrentProjet(project);
            setNbDays(project.total);
        }
    }
}
```

Dans le fichier luccastimesheet j'ai modfiié la fonction luccaTimeEntryToViewByDayItem pour sauvegarer le status :

```
const savedStatus = localStorage.getItem(`project_${projectCode}_status`);
```

Et le sauvegarder dans l'objet actuel :

```
currentProject = {
    id: projectId,
    name: projectName,
    code: projectCode,
    total: 0,
    facturable,
    factured: !!invoice,
    status: savedStatus || (invoice ? invoice.status : ''),
    fileUrl: invoice ? invoice.fileUrl : '',
    daysWorked: {},
};
```

Modification du fichier invoice.service.ts (celui du frontend/src/services) avec l'ajout de la constante invoice, pour permettre de créer une facture :

```
/**
 * Créer une nouvelle facture en BDD
 */
export const createBackendInvoice = async (
    invoiceDto: IBackendInvoiceDTO
): Promise<Invoice> => {
    const invoice: Invoice = await postBackendApi<Invoice>(
        BackendApiEndpoint.Invoice,
        invoiceDto
    );
    return invoice;
};
```

Puis j'ai modifié le fichier ModalFacturation, en tout premier cette première constante :

```
function ModalFacturation({ projects, onClose, currentMonth, user }: Props) {
    const projectOptions: SelectOption[] = projects.map((p) => {// Options pour la sélection des projets
        return { label: p.code, value: p.code };
    });

    const [currentProject, setCurrentProjet] = useState<ViewDayItemProject>(() => {
        const project = projects[0];
        // Restore status from localStorage if available
        const savedStatus = localStorage.getItem(`project_${project.code}_status`);
        if (savedStatus) {
            project.status = savedStatus;
        }
        return project;
    });
```

Puis j'ai ajouté un if dans cette fonction, pour

```
/**
 * Test & gestion si changement de projet
 */
function handleProjectChanged() {
    // changement de projet ?
    if (currentProject.code != selectedProjectCode) {
        // on cache les détails de facture
        setShowDetailInvoice(false);
        // récupération du bon projet
        const project: ViewDayItemProject | undefined = projects.find(
            (p) => p.code == selectedProjectCode
        );
        // Projet trouvé ?
        if (project) {
            // Restore status from localStorage if available
            const savedStatus = localStorage.getItem(`project_${project.code}_status`);
            if (savedStatus) {
                project.status = savedStatus;
            }
            setCurrentProjet(project);
            setNbDays(project.total);
```

Dans FacturationInvoiceDetail :

```
rationInvoiceDetail.tsx (cee241a) ↔ FacturationInvoiceDetail.tsx (8b97de3)   FacturationInvoiceDetail.tsx (8b97de3)
ers > Delia > lucca-2-penny > lucca-2-penny > lucca-2-penny > frontend > src > pages > LuccaViewByDay > modals > ModalFacturation > components > FacturationInvoiceDetail.tsx
}: Props) {

    // Générer un titre unique si une facture existe déjà
    const uniquePdfTitle = useMemo(() => {
        if (bddInvoice) {
            const timestamp = Date.now();
            return `${user.firstName.charAt(0)}${user.lastName.replace(/\s+/g, '').substring(0, 3)} - Prestation ${format(month, 'MM/yyyy')} - ${timestamp}`;
        }
        return pdfTitle;
    }, [bddInvoice, pdfTitle, user.firstName, user.lastName, month]);

    const [isDraft, setIsDraft] = useState<boolean>(invoice.draft);
    const [userHasInteractedWithDraft, setUserHasInteractedWithDraft] = useState<boolean>(false);
    const editIsDraft = (event: React.ChangeEvent<HTMLInputElement>) => {
        setUserHasInteractedWithDraft(true);
        const newIsDraft = event.target.checked;
        setIsDraft(newIsDraft);

        // Si on passe de brouillon à validé et que la date est dans le futur, la ramener à aujourd'hui
        if (!newIsDraft && invoiceDate > new Date()) {
            const today = new Date();
            setInvoiceDate(today);
            // Mettre à jour immédiatement la date dans l'objet invoice
            invoice.date = format(today, 'yyyy-MM-dd');
            // Recalculer la deadline
            const paymentDays = Math.min(+(customer.payment_conditions?.split('_')[0] || 60), 30);
            const deadline = addDays(today, paymentDays);
            invoice.deadline = format(deadline, 'yyyy-MM-dd');
            setInvoiceDeadline(deadline);
        }

        // Save status immediately to localStorage when user changes it
        const status = newIsDraft ? 'draft' : 'validated';
        localStorage.setItem(`project_${currentProject.code}_status`, status);
    };
```

```
const [invoiceDate, setInvoiceDate] = useState<Date>(() => {
    // Pour les factures validées, utiliser toujours aujourd'hui comme date
    // Pour les brouillons, utiliser la date de la facture ou aujourd'hui
    if (!invoice.draft) {
        return new Date();
    }
    const initialDate = invoice.date ? new Date(invoice.date) : new Date();
    return initialDate > new Date() ? new Date() : initialDate;
});

const [invoiceDeadline, setInvoiceDeadline] = useState<Date>(
    invoice.deadline ? new Date(invoice.deadline) : new Date()
);

const [userHasInteractedWithDate, setUserHasInteractedWithDate] = useState<boolean>(false);

// Si une facture validée existe déjà, empêcher la création d'une nouvelle facture
// Mais permettre la conversion d'un brouillon en facture validée
useEffect(() => {
    if (bddInvoice && bddInvoice.status === 'validated' && !userHasInteractedWithDraft && !userHasInteractedWithDate) {
        // Ne pas ajuster la date, mais plutôt empêcher la création
        // L'utilisateur verra le message d'avertissement et devra décider quoi faire
    }
}, [bddInvoice, userHasInteractedWithDraft, userHasInteractedWithDate]);

const changeDate = (date: Date | null) => {
    setUserHasInteractedWithDate(true);
    if (date) setInvoiceDate(date);
};
```

```
useEffect(() => {
    const today = new Date();
    let finalInvoiceDate = invoiceDate;

    // Pour les factures validées, la date ne peut pas être dans le futur
    // Pour les brouillons, on peut avoir une date dans le futur
    if (!isDraft && invoiceDate > today) {
        finalInvoiceDate = today;
        setInvoiceDate(today);
    }

    // S'assurer que même à l'initialisation, les factures validées n'ont pas de date future
    if (!isDraft && finalInvoiceDate > today) {
        finalInvoiceDate = today;
        setInvoiceDate(today);
    }

    invoice.date = format(finalInvoiceDate, 'yyyy-MM-dd');

    const paymentDays = Math.min(+(customer.payment_conditions?.split('_')[0] || 60), 30);
    const deadline = addDays(finalInvoiceDate, paymentDays);

    invoice.deadline = format(deadline, 'yyyy-MM-dd');
    setInvoiceDeadline(deadline);
}, [invoiceDate, isDraft]);
```

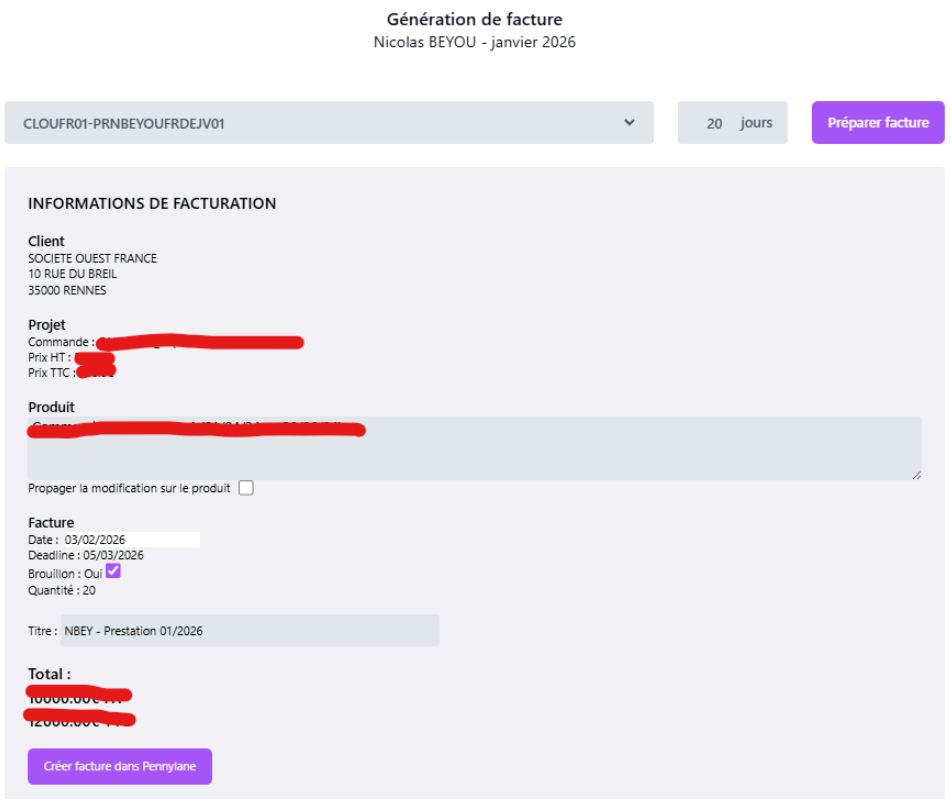
```
const [productDescription, setProductDescription] = useState<string>(product.description || '');
const editProductDescription = (event: React.ChangeEvent<HTMLTextAreaElement>) => {
    setProductDescription(event.target.value);
};

useEffect(() => {
    if (invoice.invoice_lines?.[0]) invoice.invoice_lines[0].label = productDescription;
    product.description = productDescription;
}, [productDescription]);

const [propagateToProduct, setPropagateToProduct] = useState<boolean>(false);
const editPropagateToProduct = (event: React.ChangeEvent<HTMLInputElement>) => setPropagateToProduct(event.target.checked);

useQuery(
    ['invoice', codeProject],
    () =>
        fetchInvoicesByCode(codeProject).then((datas) => {
            console.log('[FRONTEND] Fetched existing invoice:', datas[0]);
            setBddInvoice(datas[0]);
        }),
    { refetchOnWindowFocus: false }
);
```

Et principalement j'ai modifié la fonction createInvoice pour créer une facture :

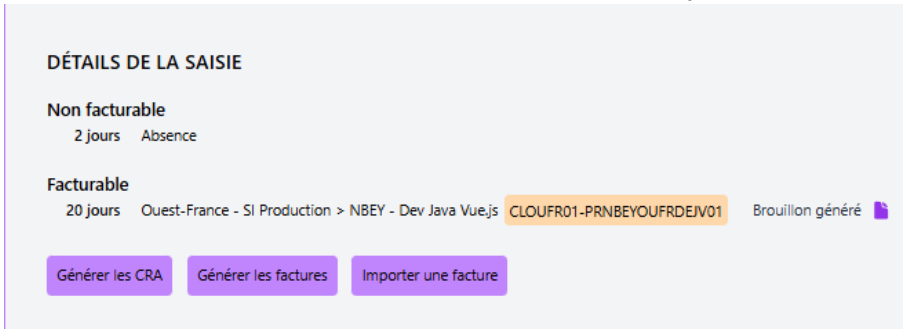
```
const createInvoice = async () => {
    setMessage('');
    setBddInvoice(null);

    try {
        // Mise à jour du produit si demandé
        if (propagateToProduct && product.external_reference) {
            await updatePennylaneProduct(product.external_reference, product);
        }

        // Si une facture brouillon existe et qu'on crée une facture validée, supprimer le brouillon et créer une nouvelle facture valide
        let invoiceToSend = invoice;
        if (bddInvoice && bddInvoice.status === 'draft' && !isDraft) {
            // Supprimer l'ancien brouillon et créer une nouvelle facture validée
            // Note: Puisque Pennylane ne permet pas de changer le statut draft via update,
            // on crée simplement une nouvelle facture validée (elle aura une date légèrement différente pour éviter les doublons)
            invoiceToSend = {
                ...invoice,
                date: invoice.date, // La date est déjà ajustée plus haut
                deadline: invoice.deadline,
                pdf_invoice_subject: uniquePdfTitle
            };
        }

        // Forcer la date à aujourd'hui pour les factures validées (pas de brouillon)
        if (!isDraft && !bddInvoice) {
            const today = format(new Date(), 'yyyy-MM-dd');
            invoiceToSend = {
                ...invoiceToSend,
                date: today,
                deadline: format(addDays(new Date(), Math.min(+(customer.payment_conditions?.split('_')[0] || 60), 30)), 'yyyy-MM-dd')
            };
        }
```

Quand on teste maintenant la fonctionnalité est réparé :

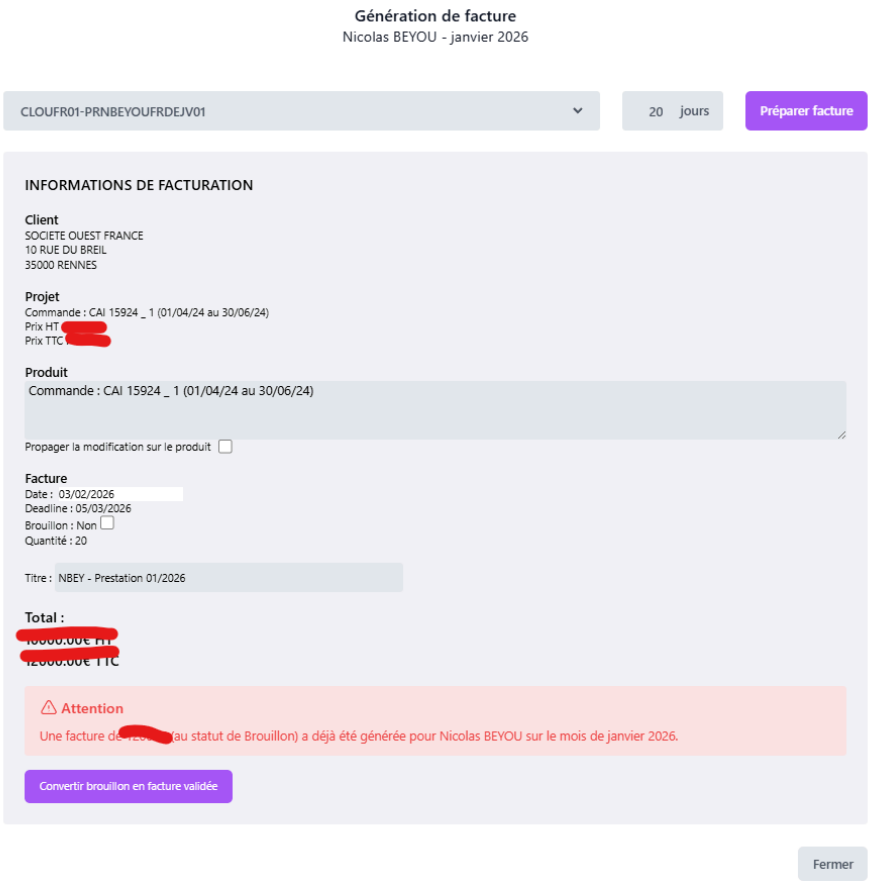
Puis on lui génère une facture :



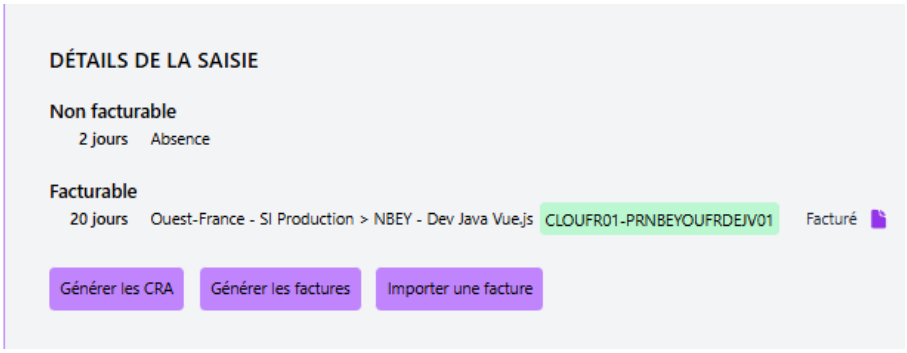
Puis qu'on fait Créer facture (ici brouillon) dans Pennylane un brouillon a bien été généré :



On fait convertir le brouillon en facture validée :

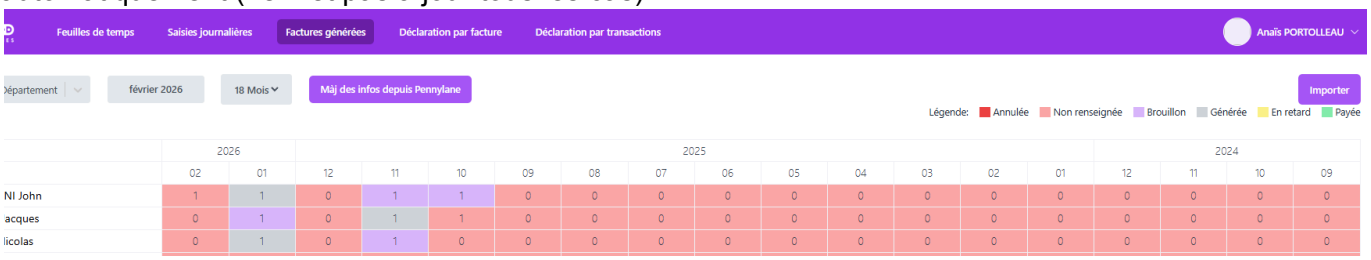


Et on peut voir que c'est bien passé en vert :

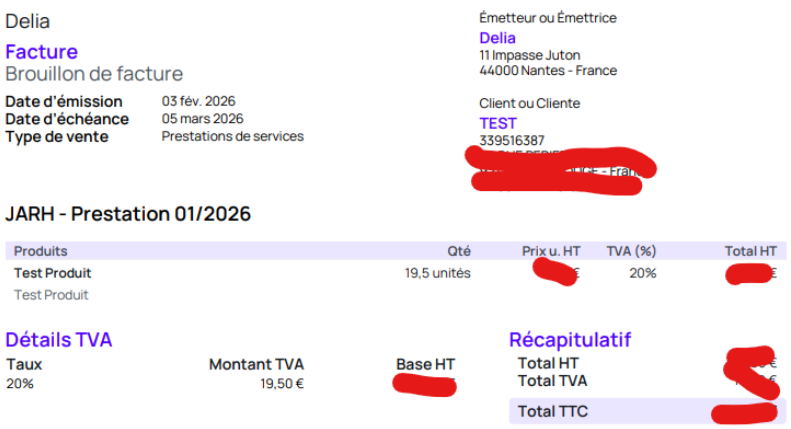


- Réparation du tableau des factures générées

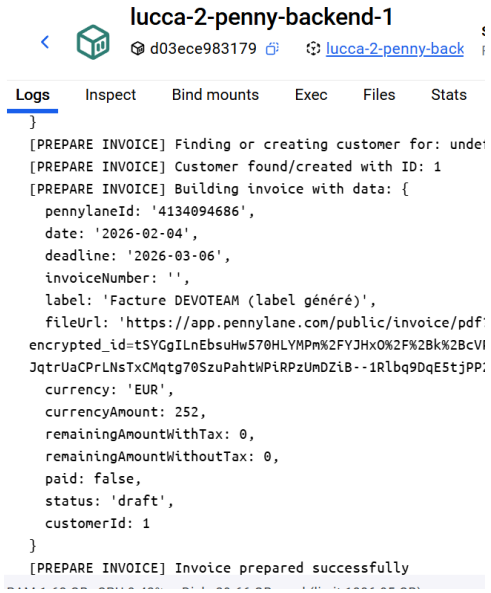
Le tableau après la réparation précédente ne marchait plus dans certains cas pour mettre à jour de brouillon à valider donc j'ai du le réparer car quand on génère cela ne se met pas à jour automatiquement (ne met pas à jour tous les cas)



Le potentiel problème que j'ai remarqué est que quand on clique sur Facturé quand elle est passée en vert cela met l'ancienne facture (c'est à dire un brouillon):



Donc, j'ai regardé dans les logs docker et j'ai trouvé pourquoi ça ne mettait pas bien à jour dans les logs Docker on peut voir que le status reste à draft même après le changement



Et plus tard cette ligne qui donne un énorme indice que les factures n'arrivent pas bien à ce mettre à jour :

```
    at fetchPennylaneInvoices (/app/src/api/pennylane/services/invoice.service.ts:45:30)
2026-02-04T09:16:03.138Z info: Received a GET request for /api/invoices?month=2026-01
Erreur critique lors de la mise à jour des factures: SyntaxError: Unexpected token 'R', "Rate limit"... is not valid JSON
    at JSON.parse (<anonymous>)
```

<u>Correction apportées :</u>

Dans le fichier <u>api.ts</u>, j'ai rajouté une fonction delete pour supprimer le brouillon existant pour le remplacer par la facture finale pour éviter d'avoir l'erreur de doublon que j'ai pu rencontré et qui ne voulait pas passer de brouillon à généré car il y avait le même id :

```typescript
/**
 * Suppression d'une ressource Pennylane
 */
export const deleteToPennylaneApi = (
    endpoint: PennylaneApiEndpoint,
    resourceId: string
) => {
    const url = `${endpoint}/${resourceId}`;

    return pennylaneAxiosInstance
        .delete(url)
        .then((response) => response.data);
};
```

Puis j'ai modifié la méthode createInvoice dans invoice.service.ts en modifiant la condition if pour penser au cas où il existe déjà une facture dans la partie backend :

```typescript
if (existingInvoiceByCode) {
    console.log(
        `[CREATE INVOICE] Invoice with code ${invoiceDto.code}
        already exists: ID: ${existingInvoiceByCode.id}, Status: ${existingInvoiceByCode.status}, PennylaneId: ${existingInvoiceByCode.pennylaneId}.
        Updating with new data.`
    );

    // Update the existing invoice with new data from Pennylane
    const updatedInvoice = await prepareInvoiceFromPennylaneObject(invoiceDto.invoice, false);
    updatedInvoice.id = existingInvoiceByCode.id;
    updatedInvoice.set({
        luccaProjectId: invoiceDto.luccaProjectId,
        month: invoiceDto.month,
        code: invoiceDto.code,
        pennylaneId: invoiceDto.invoice.id?.toString() ?? '',
        label: invoiceDto.invoice.label ?? 'Facture générée',
    });
    await updatedInvoice.save();

    // Update line items
    await LineItem.destroy({ where: { invoiceId: existingInvoiceByCode.id } });
    if (invoiceDto.invoice.invoice_lines?.length) {
        await LineItem.bulkCreate(
            invoiceDto.invoice.invoice_lines.map((item) => ({
                productId: item.product_id,
                label: item.label,
                unit: item.unit,
                quantity: +item.quantity,
                amount: +item.amount,
                invoiceId: existingInvoiceByCode.id,
            }))
        );
    }
    return await getInvoiceById(existingInvoiceByCode.id);
}
```

Et dans le même fichier mais du côté frontend, j'ai ajouté la méthode deleteExistingDraft :

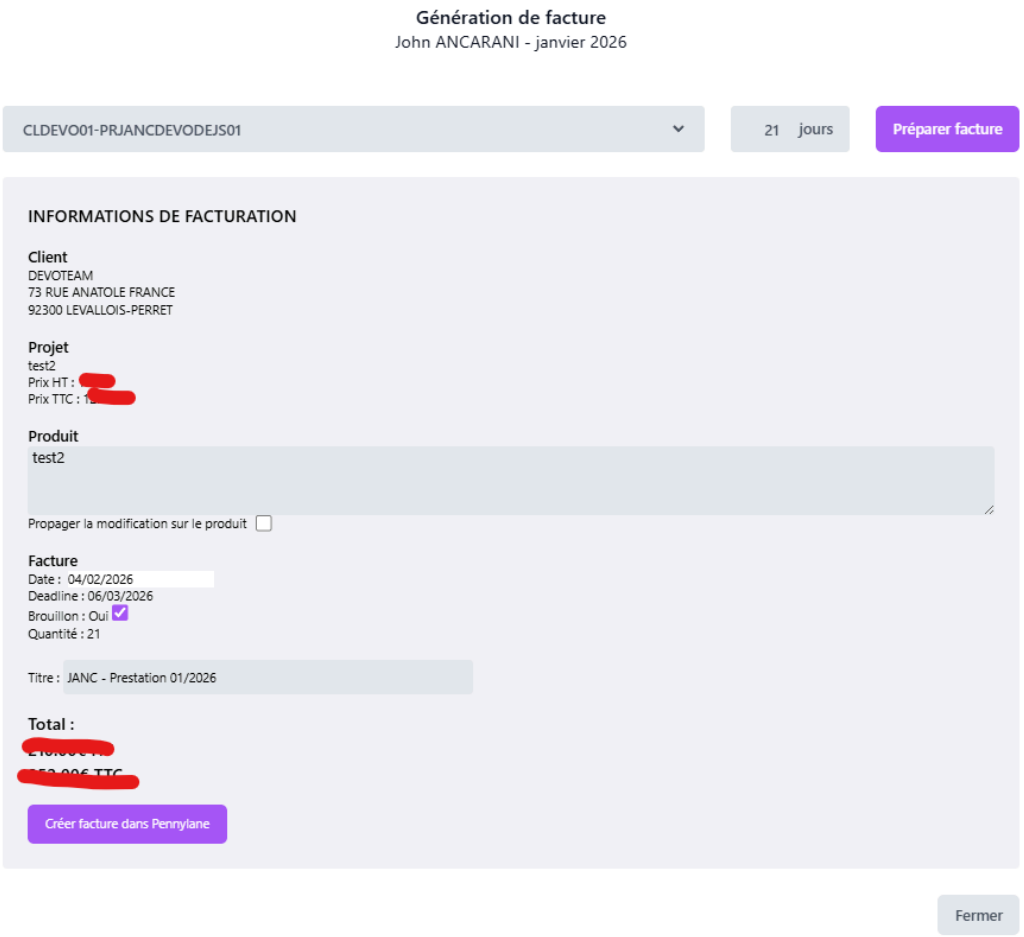
```typescript
/**
 * Supprimer le brouillon existant pour un external_reference donné
 */
export const deleteExistingDraft = async (externalReference: string): Promise<void> => {
    try {
        console.log(`[deleteExistingDraft] Recherche de brouillon pour external_reference: ${externalReference}`);

        // Filtrer par external_reference du customer
        const filters: IPennylaneFilter[] = [
            {
                field: 'customer.external_reference',
                operator: PennylaneApiOperator.equalTo,
                value: externalReference,
            },
            {
                field: 'draft',
                operator: PennylaneApiOperator.equalTo,
                value: 'true',
            },
        ];

        const fields = ['id', 'draft', 'customer.external_reference'];

        const invoices = await fetchPennylaneInvoices(null, filters, fields);

        const draftInvoice = invoices.find(inv => inv.draft === true);

        if (draftInvoice) {
            console.log(`[deleteExistingDraft] Suppression du brouillon existant: ${draftInvoice.id}`);
            await deleteToPennylaneApi(PennylaneApiEndpoint.Invoice, draftInvoice.id.toString());
            console.log(`[deleteExistingDraft] Brouillon supprimé avec succès`);
        } else {
            console.log(`[deleteExistingDraft] Aucun brouillon trouvé pour external_reference: ${externalReference}`);
        }
    } catch (error) {
        console.error(`[deleteExistingDraft] Erreur lors de la suppression du brouillon:`, error);
        // Ne pas bloquer la création de la facture si la suppression échoue
    }
};
```

J’ai rajouté cette condition dans createPennylaneInvoice pour contrôler et supprimer le brouillon existant si on a besoin de le supprimer pour créer la facture:

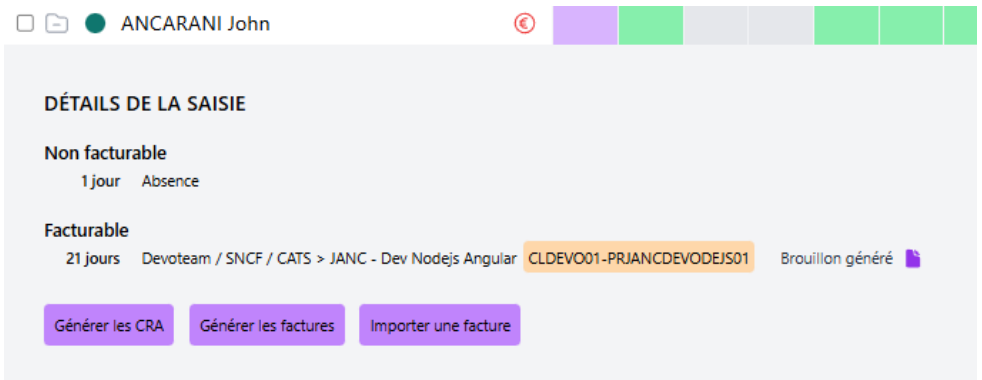
```typescript
// Pour les factures validées (draft=false), supprimer le brouillon existant avant de créer
if ('draft' in payload && payload.draft === false) {
    console.log('[createPennylaneInvoice] Facture validée détectée, suppression du brouillon existant');
    await deleteExistingDraft(payload.external_reference);
    delete payload.draft;
}
```

Et si on teste :

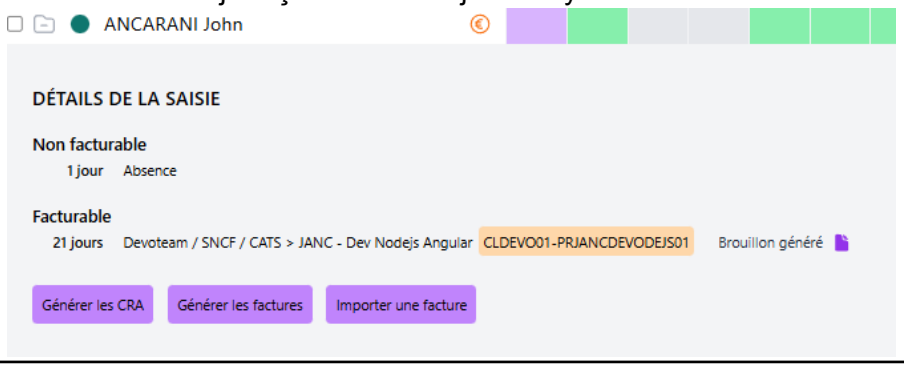
Et qu’on fait Préparer la facture :



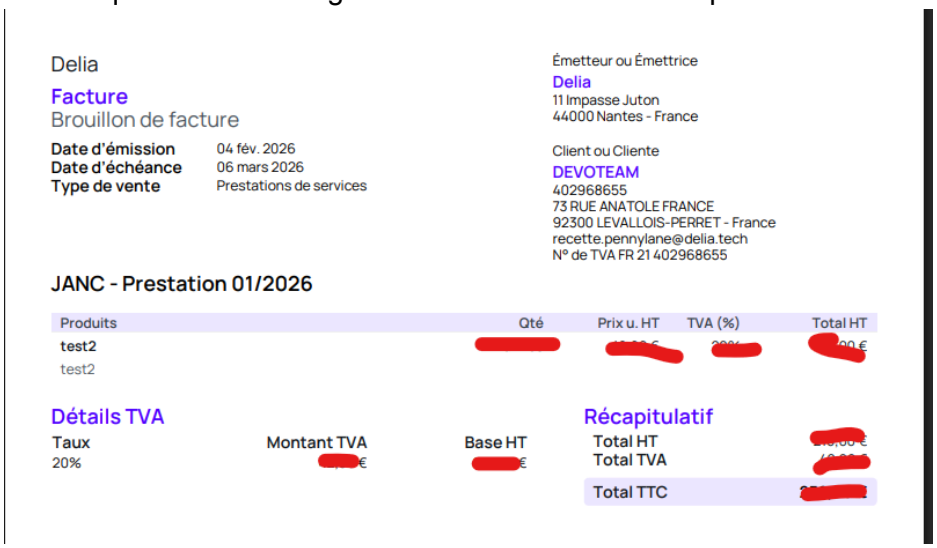
Puis on fait Créer :



Si on fait Mise à jour ça met bien à jour le symbole euro :



Si on clique sur brouillon généré c’est bien le brouillon pdf :



Delia

Facture

Brouillon de facture

Date d'émission 04 fév. 2026
Date d'échéance 06 mars 2026
Type de vente Prestations de services

Émetteur ou Émettrice
Delia
11 Impasse Juton
44000 Nantes - France

Client ou Cliente
DEVOTEAM
402968655
73 RUE ANATOLE FRANCE
92300 LEVALLOIS-PERRET - France
recette.pennylane@delia.tech
N° de TVA FR 21 402968655

JANC - Prestation 01/2026

Produits	Qté	Prix u. HT	TVA (%)	Total HT
test2 test2	[redacted]	[redacted]	[redacted]	[redacted] €

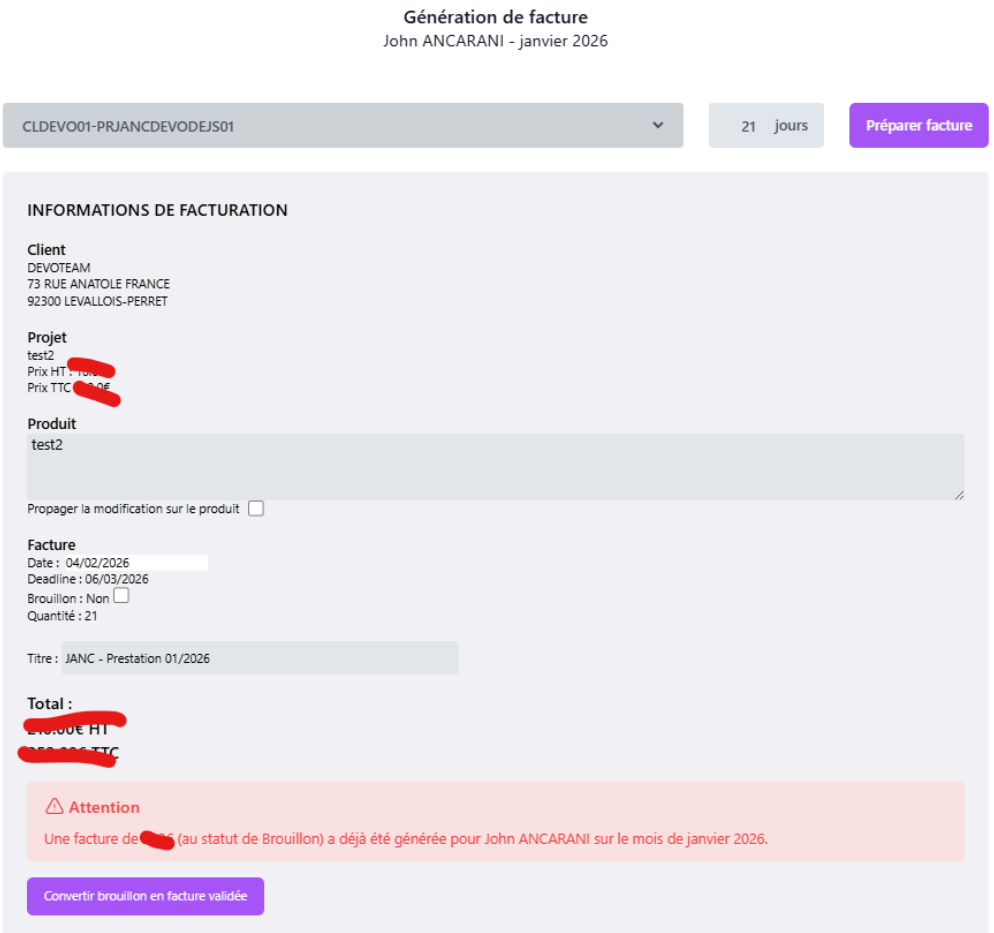
Détails TVA

Taux	Montant TVA	Base HT
20%	[redacted]€	[redacted]€

Récapitulatif

Total HT	[redacted]
Total TVA	[redacted]
Total TTC	[redacted]

Puis si on veut le passer de brouillon à générée on refait une demande :



Génération de facture
John ANCARANI - janvier 2026

CLDEVO01-PRJANCDEVODEJS01 | 21 jours | Préparer facture

INFORMATIONS DE FACTURATION

Client
DEVOTEAM
73 RUE ANATOLE FRANCE
92300 LEVALLOIS-PERRET

Projet
test2
Prix HT : [redacted]
Prix TTC [redacted]

Produit
test2

Propager la modification sur le produit ☐

Facture
Date : 04/02/2026
Deadline : 06/03/2026
Brouillon : Non ☐
Quantité : 21

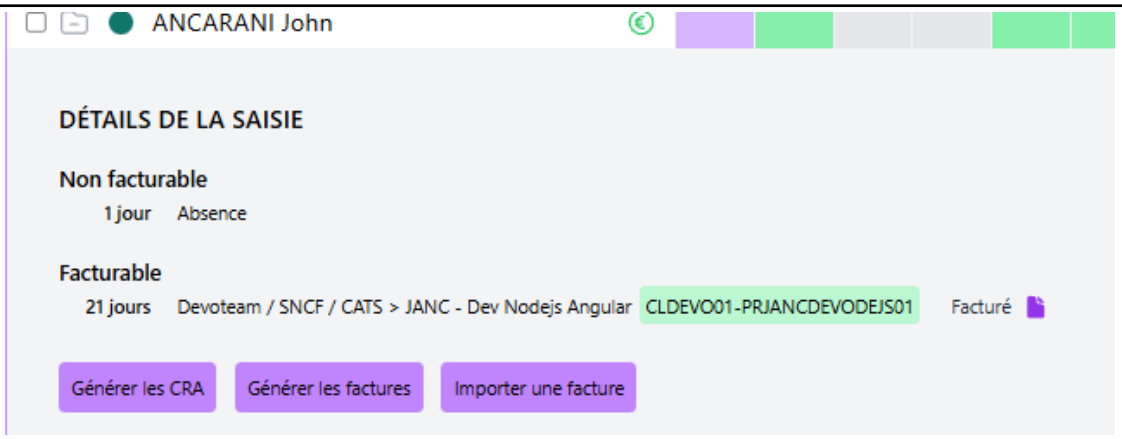
Titre : JANC - Prestation 01/2026

Total :
[redacted]

Attention
Une facture de [redacted] (au statut de Brouillon) a déjà été générée pour John ANCARANI sur le mois de janvier 2026.

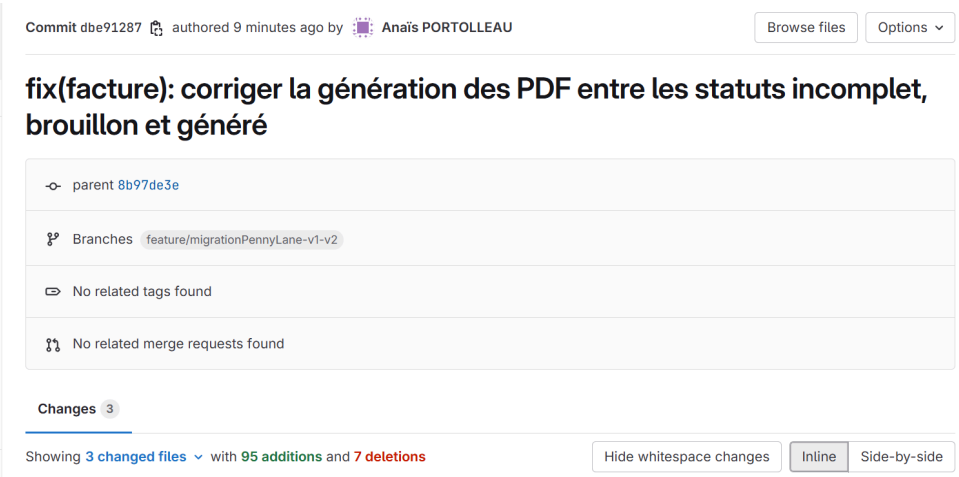
Convertir brouillon en facture validée

Puis on cliquer sur Convertir, il est bien en facturé :



Et la facture PDF est bien généré :

- Apprendre à faire des conventional commit :



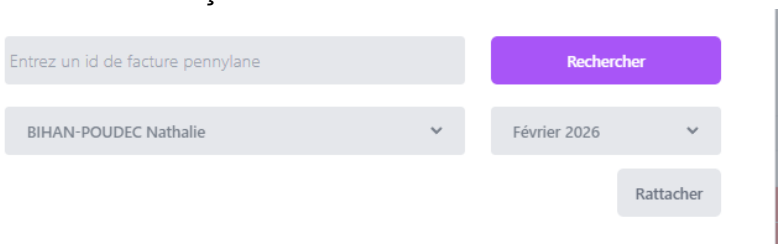
- **Liste déroulante sur la page factures** :
  - Problème :
    La liste ne sélectionne pas le bon salarié lorsqu'on clique sur une ligne.

J’ai remplacé ducoup dans le fichier ModalImportInvoice le defaultValue par cette ligne defaultValue={employeeId?.toString() || selectedUser?.id?.toString()} :

```
<div className="flex space-x-6 mt-5">
  {status === 'success' && (
    <span className="w-2/3">
      <SelectList
        options={luccaUsersToSelectUsers(allUser)}
        onChange={(selected) => selectUserById(Number(selected))}
        defaultValue={selectedUser?.id}
        disabled={!!employeeId}
      />
    </span>
  )}
  <span className="w-1/3">
    <SelectList
      options={monthOptions}
      onChange={setSelectedMonth}
      defaultValue={selectedMonth}
      disabled={!!month}
    />
  </span>
</div>
```

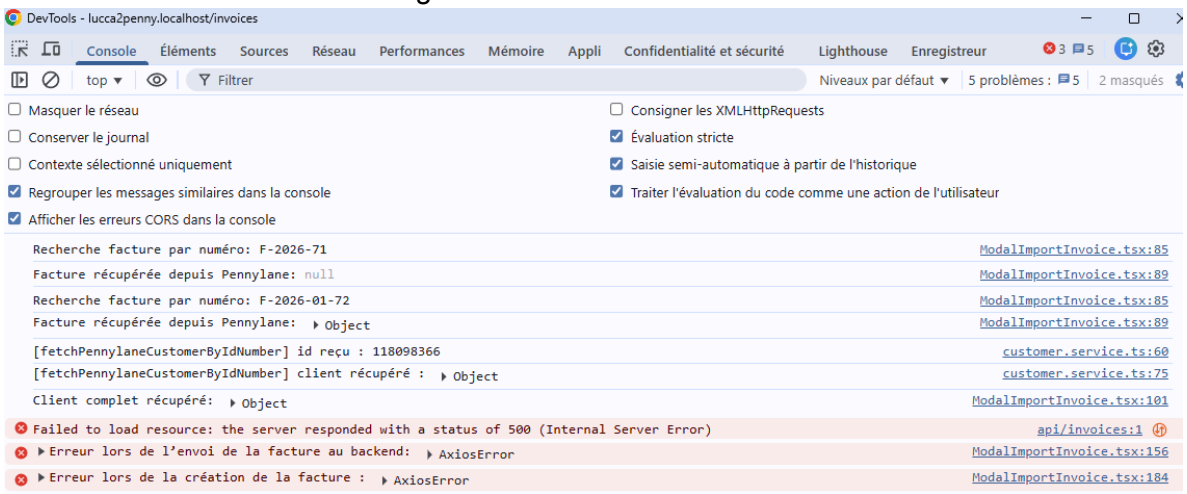
```
<div className="flex space-x-6 mt-5">
  {status === 'success' && (
    <span className="w-2/3">
      <SelectList
        options={luccaUsersToSelectUsers(allUser)}
        onChange={(selected) => selectUserById(Number(selected))}
        defaultValue={employeeId?.toString() || selectedUser?.id?.toString()}
        disabled={!!employeeId}
      />
    </span>
  )}
  <span className="w-1/3">
    <SelectList
      options={monthOptions}
      onChange={setSelectedMonth}
      defaultValue={selectedMonth}
      disabled={!!month}
    />
  </span>
</div>
```

Ensuite si on teste ça marche bien :



- Bug au niveau du rattachement des factures :

Rencontre une erreur et si on regarde dans la console :



Je ne pouvais plus importés des factures depuis Factures Générées, j’ai modifié la condition if :

```
// Créer les lignes items
if (invoiceDto.invoice.invoice_lines?.length) {
    console.log('[CREATE INVOICE] Creating line items');
    await LineItem.bulkCreate(
        invoiceDto.invoice.invoice_lines.map((item) => ({
            productId: item.product_id?.toString() || 'N/A',
            label: item.label || 'N/A',
            unit: item.unit || 'N/A',
            quantity: +item.quantity || 0,
            amount: +item.amount || 0,
            invoiceId: invoice.id,
        }))
    );
    console.log('[CREATE INVOICE] Line items created successfully');
}
```

où j'ai rajouté des conditions supplémentaires :

```
invoiceDto.invoice.invoice_lines.map((item) => ({
    productId: item.product_id?.toString() || 'N/A',
    label: item.label || 'N/A',
    unit: item.unit || 'N/A',
    quantity: +item.quantity || 0,
    amount: +item.amount || 0,
    invoiceId: invoice.id,
}))
```

pareil avec une autre condition que j'ai modifié :

```
if (invoice.invoice_lines && invoice.invoice_lines.length > 0) {
    await LineItem.bulkCreate(
        invoice.invoice_lines.map((item: IPennylaneInvoiceLineItem) => ({
            productId: item.product_id,
            label: item.label,
            unit: item.unit,
            quantity: +item.quantity,
            amount: +item.amount,
            invoiceId: bddInvoice.id,
        }))
    );
}
```

par :

```
if (invoice.invoice_lines && invoice.invoice_lines.length > 0) {
    await LineItem.bulkCreate(
        invoice.invoice_lines.map((item: IPennylaneInvoiceLineItem) => ({
            productId: item.product_id?.toString() || 'N/A',
            label: item.label || 'N/A',
            unit: item.unit || 'N/A',
            quantity: +item.quantity || 0,
            amount: +item.amount || 0,
            invoiceId: bddInvoice.id,
        }))
    );
}
```

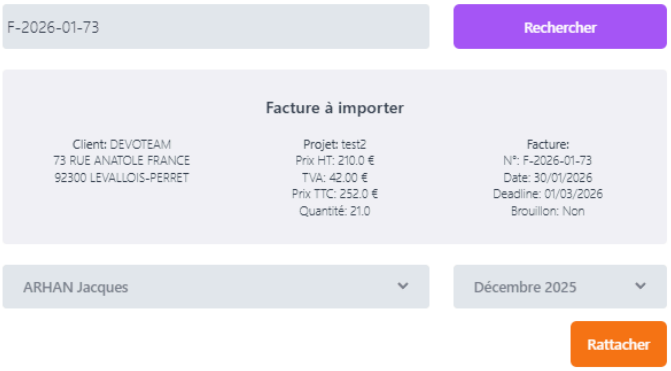
Et pareil encore pour cette condition :

```
if (
    pennylaneInvoice.invoice_lines &&
    pennylaneInvoice.invoice_lines.length > 0
) {
    await LineItem.bulkCreate(
        pennylaneInvoice.invoice_lines.map(
            (item: IPennylaneInvoiceLineItem) => ({
                productId: item.product_id,
                label: item.label,
                unit: item.unit,
                quantity: +item.quantity,
                amount: +item.amount,
                invoiceId: invoice.id,
            })
        )
    );
    console.log(
        `[REFRESH INVOICES] Lignes items créées pour
    );
}
```

par :

```
if (
    pennylaneInvoice.invoice_lines &&
    pennylaneInvoice.invoice_lines.length > 0
) {
    await LineItem.bulkCreate(
        pennylaneInvoice.invoice_lines.map(
            (item: IPennylaneInvoiceLineItem) => ({
                productId: item.product_id?.toString() || 'N/A',
                label: item.label || 'N/A',
                unit: item.unit || 'N/A',
                quantity: +item.quantity || 0,
                amount: +item.amount || 0,
                invoiceId: invoice.id,
            })
        )
    );
```

Si on teste cela marche bien :



Et si on fait Rechercher cela affiche bien les infos de la facture comme on peut le voir ci-dessus.

Puis si on fait “Rattacher” :

5. Réunions

- Réunion ave pour faire un point sur mon CV et mon Linkedin
- Réunion avec mon tuteur Julien Relandeau pour faire le point sur le projet et nous donner des consignes par rapport à la futur migration & point sur le rapport de stage

6. Préparation de la migration

- J’ai un très gros travail à faire sur le nettoyage de code pour prévoir un éventuel déploiement de l’application

Observations éventuelles :

Cette semaine a mis en évidence plusieurs limites dans la gestion des factures, notamment concernant les différents états (brouillon, générée, validée) qui n’étaient pas toujours correctement synchronisés entre le frontend et le backend. Cela a entraîné des incohérences d’affichage, des difficultés de rattachement et des risques de doublons. Les corrections apportées ont permis de stabiliser la majorité des cas.

Objectifs pour la semaine suivante *(obligatoire)* :

Poursuivre le nettoyage et la sécurisation du code en vue de la migration de l’application.
Tester l’ensemble des parcours utilisateurs autour de la génération, l’importation et la mise à jour des factures.
Avancer sur la préparation du rapport de stage et intégrer les retours du tuteur.